

Long Text to Predictive Features: LLM-Guided Blockwise Feature Engineering via Executable Program Search

Ziming Dai*
City University of Hong Kong
Hong Kong, China
phoenix.dai@my.cityu.edu.hk

Dabiao Ma*
Qfin Holdings, Inc.
Beijing, China
madabiao-jk@qifu.com

Ziheng Guo
Tianjin University
Tianjin, China
sygzh6@tju.edu.cn

Jack Dong
Carnegie Mellon University
Pittsburgh, USA
jack.dong@bytedance.com

Zimu Zhou[✉]
City University of Hong Kong
Hong Kong, China
zimuzhou@cityu.edu.hk

Abstract

Industrial risk-control systems typically rely on structured-data models for efficient prediction, yet substantial valuable information remains embedded in unstructured long text. Extracting this information through manual feature engineering is labor-intensive, while requiring a large language model (LLM) to process every real-time input may not meet practical deployment requirements. To address this challenge, we propose LLM-BlockFE, an LLM-guided offline feature construction framework that converts long text into executable feature programs, thereby avoiding LLM calls during online inference. LLM-BlockFE constructs feature programs by incrementally appending immutable code blocks and evaluates candidate features using a downstream model. To address the tendency of conventional greedy search to become trapped in suboptimal solutions, our method introduces a block-level rollback mechanism based on depth-calibrated credit allocation and advances multiple independent search trajectories in an interleaved manner, reducing redundant exploration by sharing fixed descriptions of each trajectory’s exploration direction. After the search, the resulting programs are frozen and deployed to extract structured features for downstream prediction models. Across two public and two private datasets, LLM-BlockFE achieves absolute AUC improvements of 0.0069 to 0.0358 over the strongest baseline on each dataset in the full-dataset comparison. Post-launch monitoring across five deployed financial risk-control applications shows absolute KS improvements of 0.02 to 1.56 percentage points over the existing manually designed strategy.

Keywords

Automated Feature Engineering, Large Language Models, Unstructured Long Text, Program Synthesis, Interpretability

1 Introduction

In high-stakes industrial applications such as financial risk control, predictive models must not only achieve high accuracy but also meet stringent constraints on inference latency and maintainability. Gradient-boosted tree models for structured data, such as XGBoost [7] and LightGBM [17], therefore remain widely used in industry because of their computational efficiency and seamless

compatibility with existing business systems. Meanwhile, as business operations become more complex, unstructured long texts, such as transcripts of customer service calls, are increasingly becoming an important source of business information [16]. These texts contain rich information about user behavior, actual intent, and latent risk signals, but their unstructured form makes them difficult to use directly with commonly adopted structured-data models [12]. Consequently, extracting effective features from raw long text has become a key bottleneck in industrial modeling.

At present, enterprises primarily rely on domain experts to design cleaning rules, text parsing logic, and feature computation programs, and then standardize the extracted outputs for use by downstream models. Variations such as colloquial language, omitted expressions, paraphrases, and cross-sentence semantics require manually designed parsing logic and feature rules to be continually extended and maintained. Existing text representation methods also have limitations: shallow features such as TF-IDF have limited capacity to represent contextual semantics. Dense vectors produced by pretrained models are difficult to interpret directly. Although long-sequence encoders can model long text context, their outputs typically require further processing to form structured feature columns with clear semantics [3, 5, 30]. In addition, automated feature engineering methods that take existing structured feature columns as input primarily search for transformations and combinations of variables, without directly addressing feature construction from raw long text [31].

In recent years, large language models (LLMs) have demonstrated strong natural language understanding and code generation capabilities, offering a new technical approach to feature engineering [22, 26]. Some studies use LLMs to generate feature transformation code for existing tabular columns [1], while others use LLMs to extract features with clear semantics from text [20]. For methods that rely on an LLM to generate feature values for individual samples, continuously arriving text inputs that require immediate scoring still need LLM inference, even when the feature definitions are established offline. This increases the latency and computational overhead of online feature extraction, placing pressure on industrial deployments with high concurrency.

To address these deployment constraints, we propose **LLM-BlockFE**, which formulates long text feature construction as the offline synthesis of executable feature programs. Unlike approaches that directly invoke an LLM for end-to-end prediction or per-sample

*Both authors contributed equally to this research.

feature extraction, LLM-BlockFE uses the LLM’s semantic reasoning and code generation capabilities only during the offline stage to discover features and write the corresponding extraction programs. The resulting structured features are then provided to a downstream supervised model. This design allows the feature logic to be reviewed and maintained in code form and requires no LLM calls during the online stage.

Specifically, LLM-BlockFE constructs scalar feature extraction programs for long text and uses Incremental Program Search to generate a feature set whose members jointly improve downstream model performance. During the search, our method introduces a block-level rollback mechanism based on depth-calibrated credit allocation and advances multiple independent feature construction trajectories in an interleaved manner, reducing redundant exploration by sharing fixed descriptions of each trajectory’s exploration direction. In deployment, the business system only needs to execute the finalized feature program code and its lightweight dependencies, with zero online LLM calls. Across five real-world financial risk-control applications, post-launch monitoring shows absolute KS improvements of 0.02 to 1.56 percentage points over the existing manually designed strategy across all 15 application-month comparisons. In addition, the offline evaluation results show that our method outperforms the strongest competing baseline on AUC values across all four datasets.

The contributions of this paper are summarized as follows:

- We propose LLM-BlockFE for long text feature engineering, formulating feature construction as the offline synthesis of executable programs. We design a protocol for complete program generation and immutable block-level updates, providing explicit operational boundaries for incremental construction and local rollback. The resulting programs are used directly for online feature extraction, without requiring online LLM calls.
- We propose a depth-calibrated credit-guided backtracking mechanism that separately evaluates the current program’s predictive performance and its opportunity for further expansion. Based on a node’s recent expansion outcomes and historical baselines at the same depth, the mechanism determines whether to continue expansion or restore the parent program, reducing the dependence of single-path search on early choices.
- Experiments on multiple public and private datasets demonstrate that our method outperforms state-of-the-art (SOTA) baselines. Deployment and online evaluation in five real-world financial risk-control applications further validate the improvements that LLM-BlockFE brings to downstream predictive performance and its value for industrial applications.

2 Related Work

Traditional Long Text Representation and Feature Engineering. Traditional long text feature engineering typically transforms raw text into representations that can be processed by models. Common approaches include using TF-IDF to encode terms and their frequencies, extracting manually defined attributes as text statistics, and further compressing sparse term representations using

dimensionality reduction methods such as SVD [24]. Another line of work uses pretrained language models to produce contextual representations. Long-sequence Transformers such as Longformer and BigBird extend the text lengths that models can process, and their encoded outputs can also be pooled to form document-level representations [5, 30].

Although the above methods can be used for text prediction, they remain insufficient to meet industrial requirements for feature controllability and interpretability. Traditional methods based on terms or manually defined rules have limited adaptability to variations in expression, such as colloquial language and omissions. Although pretrained language models can capture context, their outputs are typically high-dimensional dense vectors with no directly interpretable semantics, and require additional representation processing and inference computation. Therefore, the problem of automatically constructing compact, interpretable, and easily deployable structured features from raw long text remains unaddressed directly.

Automated Feature Engineering. Automated feature engineering typically starts from existing structured variables and searches for feature transformations that benefit downstream prediction. Methods such as OpenFE generate and select tabular features from a predefined operator space [31]. Recent work further draws on the semantic knowledge of LLMs to propose candidate transformations. CAAFE generates feature engineering code based on dataset descriptions and table columns [15], while OCTree converts predictive feedback expressed by a decision tree into natural-language reasoning to iteratively refine feature-generation rules [21]. LLM-FE formulates tabular feature transformation as program search and iteratively optimizes candidate programs through evolutionary-style search [1]. FAMOSE, in turn, adopts a ReAct-based agent workflow for feature discovery [6]. Furthermore, NSR-Boost corrects the prediction residuals of industrial legacy models by gradually generating and verifying executable expert programs [9].

However, these methods primarily search over existing tabular columns and their descriptions, with the central task of transforming or combining structured variables. This paper instead focuses on unstructured long text inputs: the system must construct features from the text content that can be used directly by tabular models, rather than merely transforming existing tabular columns.

LLM-Based Feature Engineering for Unstructured Text. Recently, several studies have begun to explore feature engineering for long text using the semantic knowledge of LLMs. FELIX uses an LLM to generate interpretable features for text samples and applies them to downstream classification [20]. Balek *et al.* study the extraction of a small number of semantic features from scientific literature, showing that these features can support text classification while remaining interpretable [3]. Barlier *et al.* investigate feature generation for joint prediction from text and tabular data [4].

However, in these methods, candidate feature values still need to be extracted or assigned by an LLM for individual text samples. Batch processing or offline precomputation can mitigate this overhead, but for new samples that arrive continuously and require immediate scoring, feature extraction still incurs additional inference costs and latency, and creates a dependency on the serving infrastructure. In contrast, LLM-BlockFE uses an LLM to construct feature programs offline. After deployment, the system generates

features by directly executing the frozen programs, without calling an LLM, thereby enabling fast inference.

3 Notations and Problem Formulation

Consider a supervised learning task with long text data, represented by the dataset:

$$\mathcal{D} = \{(u_i, \mathbf{s}_i, y_i)\}_{i=1}^N, \quad (1)$$

where $u_i \in \mathcal{U}$ denotes an unstructured long text input commonly encountered in industry, such as user dialogues, credit reports, application descriptions, or customer service records. $\mathbf{s}_i \in \mathbb{R}^{d_s}$ denotes optional pre-existing structured features, and $y_i \in \{0, 1\}$ is a binary classification label. Although this paper primarily focuses on binary financial risk prediction tasks, the proposed feature engineering algorithm can also be extended to other supervised learning settings.

LLM-BlockFE aims to automatically construct compact and predictive structured features from long text, providing downstream models with effective information beyond that contained in pre-existing structured variables. Given text u and structured features $\mathbf{s}$, the system transforms the text into a feature representation $\mathbf{z}(u)$ and makes predictions using the augmented input:

$$\hat{y} = g_\theta([\mathbf{s}, \mathbf{z}(u)]). \quad (2)$$

Here, g_θ is a supervised learning model that handles structured inputs. We use XGBoost as the primary downstream model for exposition and experiments.

Let $\mathcal{P}$ denote the space of executable text feature programs that satisfy predefined input–output interfaces and require no LLM calls at deployment. Any program $\Phi \in \mathcal{P}$ maps long text to a scalar feature, i.e., $\Phi : \mathcal{U} \rightarrow \mathbb{R}$. For a program set $\mathcal{F} = \{\Phi_1, \dots, \Phi_m\} \subseteq \mathcal{P}$, the corresponding text feature vector is:

$$\mathbf{z}_{\mathcal{F}}(u) = [\Phi_1(u), \dots, \Phi_m(u)]. \quad (3)$$

Below, $\theta(\mathcal{F})$ denotes the model parameters fitted using $[\mathbf{s}, \mathbf{z}_{\mathcal{F}}(u)]$ under the prescribed training protocol, with $\theta(\Phi)$ used as shorthand for the single-program case. $\mathcal{M}$ denotes the predictive performance metric, for which we use ROC-AUC. $\mathcal{D}_{\text{val}}$ is used for candidate comparisons during the search, while an independent test set is used for final performance evaluation. Detailed data usage is described in Section 4.

Program construction involves searching over both discrete code structures and numerical parameters. Moreover, the text signals extracted by different programs may be redundant or complementary [18]. We therefore describe the feature engineering objective at two levels: individual program construction and the combination of multiple programs.

P1. Conditional text feature-program construction. Given pre-existing structured variables $\mathbf{s}$, the objective is to construct a new feature program Φ in the program space $\mathcal{P}$ that extracts predictive information from long text not yet covered by the current feature set:

$$\Phi^* = \arg \max_{\Phi \in \mathcal{P}} \mathcal{M}(g_{\theta(\Phi)}([\mathbf{s}, \Phi(u)]); \mathcal{D}_{\text{val}}). \quad (4)$$

When the structured-feature baseline and evaluation protocol are fixed, maximizing this performance is equivalent to maximizing the predictive gain over the baseline. **P1** measures the predictive value of a single text feature program conditioned on pre-existing

structured variables, without including text features from other trajectories in the evaluation.

P2. Multi-program construction under a feature budget. Under a finite feature budget, the objective is to construct multiple text feature programs whose outputs achieve strong downstream predictive performance when used jointly with pre-existing structured variables:

$$\mathcal{F}^* = \arg \max_{\substack{\mathcal{F} \subseteq \mathcal{P} \\ |\mathcal{F}| \leq K_{\text{max}}}} \mathcal{M}(g_{\theta(\mathcal{F})}([\mathbf{s}, \mathbf{z}_{\mathcal{F}}(u)]); \mathcal{D}_{\text{val}}). \quad (5)$$

Here, K_{max} is the maximum number of text feature programs retained and deployed, constraining the dimensionality of the added features. Since a program’s marginal value depends on the information extracted by other programs in the set, combining programs obtained by independently solving **P1** does not necessarily yield the optimal set for **P2**.

LLM-BlockFE uses **P1** as the independent search objective for each trajectory. Fixed exploration directions and shared direction descriptions encourage the exploration of different text signals. After the search, the system combines the best programs archived by the individual trajectories and trains the final downstream model. This procedure approximates **P2**: direction sharing reduces redundant exploration, while the actual complementarity among programs and the gains from their combination are evaluated empirically.

4 Methodology

As shown in Figure 1, the LLM-BlockFE architecture consists of three tightly coupled components. First, the system generates a complete candidate program and extracts the added code block using predefined markers, enabling incremental updates while keeping existing code unchanged. Parameters within the added block are then locally tuned through Bayesian optimization. Second, a depth-calibrated credit mechanism evaluates the relative opportunity of obtaining an acceptable child through further expansion of the current program node, guiding continued expansion or dynamic backtracking. Finally, guided by fixed exploration directions, the system advances multiple search trajectories asynchronously in an interleaved manner and shares direction descriptions to reduce redundant exploration. After the search, the system collects and freezes the best programs from all trajectories for online feature extraction, without requiring online LLM calls.

4.1 Downstream-Guided Block Construction

To construct a single text feature program as defined in Section 3, LLM-BlockFE starts from the Seed Program P_0 , which specifies the input, output, and code insertion point. P_0 defines only the available libraries and the standardized output. Consider the n -th search trajectory. The current program consists of the initial program and the code blocks accepted along the current path:

$$P_n = P_0 \oplus b_{n,1} \oplus \dots \oplus b_{n,m_n}, \quad (6)$$

where $\oplus$ is appending a code block at the designated location, and m_n is the number of code blocks on the current path. The program maps a long text input u to a scalar feature, i.e., $\Phi_n(u) = P_n(u)$.

To maintain coherence in the overall program logic while allowing local updates to be reversed, LLM-BlockFE adopts a program update mechanism based on “complete generation and incremental

OFFLINE: PROGRAM SEARCH + MODEL TRAINING

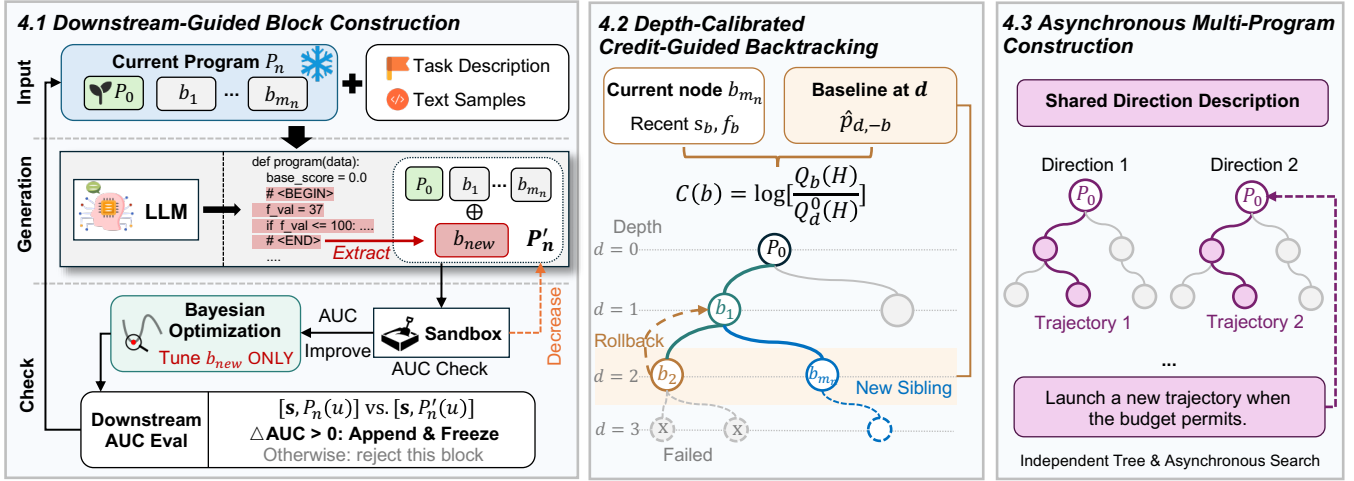


Figure 1: Overview of LLM-BlockFE, which searches for text feature programs offline and executes the selected programs for online prediction.

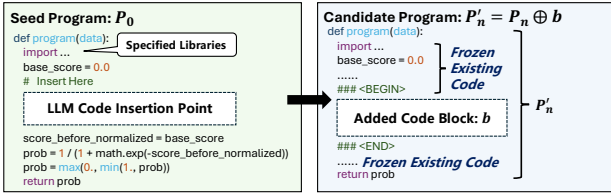


Figure 2: Illustration of the Seed program and LLM-generated candidate program.

extraction”, as shown in Figure 2. Directly requesting an isolated code addition from the LLM can readily lead to mismatches, such as variable name conflicts, calls to undefined interfaces, or semantic discontinuities with existing code, because of the model’s limited awareness of the surrounding program. LLM-BlockFE therefore uses the complete program as the unit of generation. The LLM receives P_n , a very small set of text samples $\mathcal{D}_{\text{llm}} \subset \mathcal{D}_{\text{train}}$, the task description, callable tools, and previous generation summaries. It generates complete candidate code, marks the added region using predefined markers (`<BEGIN>` . . . `<END>`), and provides a summary of the current round for subsequent iterations. The system then precisely extracts the added code block b using a regular expression, constructs the candidate program $P'_n = P_n \oplus b$, and strictly checks whether the existing code outside the markers has changed. Any candidate that impermissibly modifies the original code is rejected. Generating with the full program context promotes syntactic and logical compatibility between the new and existing code, while

block-level extraction preserves clear boundaries between them and provides the basis for subsequent local backtracking.

The candidate program first undergoes an execution check in a restricted environment. Candidates that fail to execute or produce invalid outputs are immediately discarded. For valid candidates, the system compares the AUCs of the text features produced by P'_n and P_n . If the untuned candidate decreases this AUC, it is discarded early. This step serves as a low-cost heuristic screening procedure to reduce the overhead of subsequent parameter optimization.

For candidates that pass the initial screening and contain tunable parameters, the LLM identifies the parameters to optimize, their search ranges, and the optimization objective based on the added block. Because operations such as threshold tests make usable gradients unavailable for the program output and its AUC with respect to the parameters, the system uses Bayesian optimization and writes the resulting parameter values back into the added block. Optimization affects only the added block. The existing code and its frozen parameters remain unchanged. The program with the updated parameter values must pass the execution check again and is still denoted by P'_n below.

Finally, the system uses a lightweight XGBoost model with a fixed configuration to evaluate the candidate’s downstream value. To prevent data leakage and overfitting during the search, the evaluation procedure strictly separates data usage: the lightweight XGBoost model is fitted only on a training subset $\mathcal{D}_{\text{fit}} \subset \mathcal{D}_{\text{train}}$, while performance is evaluated on an independent validation set $\mathcal{D}_{\text{val}}$. The system uses $[s, P_n(u)]$ and $[s, P'_n(u)]$, respectively, as inputs to fit the evaluation models $g_{\theta(P_n)}$ and $g_{\theta(P'_n)}$ on $\mathcal{D}_{\text{fit}}$, and then computes their AUCs on $\mathcal{D}_{\text{val}}$. The conditional gain of the added

block is defined as:

$$\Delta_{n,b} = \mathcal{M}(g_{\theta(P'_n)}([s, P'_n(u)]; \mathcal{D}_{\text{val}}) - \mathcal{M}(g_{\theta(P_n)}([s, P_n(u)]; \mathcal{D}_{\text{val}}), \quad (7)$$

where $\mathcal{M}$ denotes AUC. For a new trajectory in which no code blocks have yet been accepted, the parent model uses only the structured features s . When $\Delta_{n,b} > 0$, the system accepts and freezes the added block and appends it to the current path.

Once feature extraction by all programs is complete, the resulting feature matrix is combined with the structured features to train the final downstream risk-control model on $\mathcal{D}_{\text{down}} \subseteq \mathcal{D}_{\text{train}}$. Its generalization performance is then evaluated on an independent test set $\mathcal{D}_{\text{test}}$.

4.2 Depth-Calibrated Credit-Guided Backtracking

The previous subsection introduced candidate code-block construction, but the append-only constraint does not, by itself, determine how the program space is explored. If the search always expands along the current path, blocks accepted early may constrain subsequent exploration. NSR-Boost uses an annealed threshold to allow small performance decreases early in the search, but still uses the current program as the starting point for subsequent iterations [9]. LLM-BlockFE introduces depth-calibrated credit-guided backtracking, using the current node’s expansion opportunity to determine whether to continue the search or restore the parent program to explore an alternative branch [23, 29].

Specifically, each search trajectory can be represented as a search tree rooted at P_0 . Each non-root node is uniquely identified by its terminal code block b and corresponds to the complete program formed by accumulating code blocks from the root. The node depth d is defined as the number of accepted code blocks on its path. A node’s downstream AUC reflects the predictive performance of the current program, while its credit measures the likelihood of obtaining a valid child through further expansion from that node. For the t -th eligible expansion attempt at node b , we assign $y_{b,t} = 1$ if the candidate passes all predictive gates and is accepted, and $y_{b,t} = 0$ if it fails the initial text AUC screening, local parameter optimization, or the downstream AUC gate. Non-predictive failures, such as syntax or execution errors, are excluded from these statistics.

Because the difficulty of adding code varies substantially across depths, we construct a baseline from historical observations at the same depth [11]. Let block b have depth d , and let S_{-b} and F_{-b} denote the lifetime numbers of successes and failures, respectively, among all other non-root blocks. The global success-rate estimate is:

$$\bar{p}_{-b} = \begin{cases} p_0, & S_{-b} + F_{-b} = 0, \\ \frac{S_{-b} + \frac{1}{2}}{S_{-b} + F_{-b} + 1}, & \text{otherwise,} \end{cases} \quad (8)$$

where p_0 is a fixed cold-start success rate, and Jeffreys smoothing is used when historical observations are available [13]. Further, let $S_{d,-b}$ and $F_{d,-b}$ denote the lifetime numbers of successes and failures, respectively, among other blocks at the same depth. The

depth-specific success rate is estimated using empirical-Bayes partial pooling as [2]:

$$\hat{p}_{d,-b} = \frac{S_{d,-b} + \kappa \bar{p}_{-b}}{S_{d,-b} + F_{d,-b} + \kappa}, \quad (9)$$

where κ controls the degree of shrinkage of within-depth observations toward the global baseline. The performance of block b is modeled using recent valid attempts to adapt to changes in candidate quality and the remaining search space during the search. Let s_b and f_b denote the numbers of successes and failures, respectively, for block b in the recent window. Under a Beta-Bernoulli model, its Beta posterior parameters are defined as:

$$\alpha_b = \kappa \hat{p}_{d,-b} + s_b, \quad \beta_b = \kappa(1 - \hat{p}_{d,-b}) + f_b. \quad (10)$$

Observations outside the recent window are retained in the lifetime statistics and used to estimate the global and depth-specific baselines for other blocks. Let H denote the short-term prediction horizon. Based on the posterior distribution of block b , its predicted probability of producing at least one valid child in the next H direct attempts is:

$$Q_b(H) = 1 - \prod_{i=0}^{H-1} \frac{\beta_b + i}{\alpha_b + \beta_b + i}. \quad (11)$$

For a block at the same depth with no observations of its own, this probability is:

$$Q_d^0(H) = 1 - \prod_{i=0}^{H-1} \frac{\kappa(1 - \hat{p}_{d,-b}) + i}{\kappa + i}. \quad (12)$$

LLM-BlockFE defines block credit as the log ratio of a block’s expansion opportunity to the baseline at the same depth:

$$C(b) = \log R_b, \quad R_b = \frac{Q_b(H)}{Q_d^0(H)}. \quad (13)$$

Thus, $C(b) = 0$ indicates that the block’s opportunity matches the same-depth baseline. $C(b) > 0$ indicates an opportunity above the baseline. $C(b) < 0$ indicates an opportunity below it. A new block has no observations of its own and is therefore initialized with $C(b) = 0$. A successful block updates only the posterior of its direct parent.

When the relative opportunity R_b of the current block b falls below the threshold ρ , LLM-BlockFE rolls back one terminal block. After rollback, the search resumes from the restored parent and explores a new sibling. Since the root has no block to roll back, it is not assigned ordinary credit.

4.3 Asynchronous Multi-Program Construction

To construct multiple text feature programs, LLM-BlockFE advances multiple search trajectories in an interleaved manner. Each trajectory starts from the initial program P_0 and separately maintains its current search path, code-block history, rollback state, and best-program archive. Once a trajectory’s first code block is accepted, the LLM establishes a fixed exploration direction based on the feature semantics of that block. The system can then initiate new trajectories when the budget permits, without waiting for existing trajectories to complete their searches.

Trajectories share only descriptions of their exploration directions. When generating a candidate code block, the LLM continues

feature construction based on the current program and fixed direction of its own trajectory, while referring to the direction descriptions of other trajectories to avoid repeatedly exploring the same targets [14]. The program code, best paths, and feature outputs of other trajectories are excluded from the generation context of the current trajectory. The fixed direction constrains the primary exploration target but allows different trajectories to use the same keywords or tools.

Each trajectory follows the candidate construction and evaluation procedure in Section 4.1, independently performing initial text AUC screening, local Bayesian optimization, and downstream AUC evaluation. Downstream evaluation uses only the pre-existing structured features and the program output of the current trajectory, excluding features from other trajectories. Accepted code blocks are added to the current path. When the resulting downstream AUC exceeds the trajectory’s historical maximum, its best-program archive is updated. Rollback changes only the current path. It neither deletes the archived best program nor changes the fixed exploration direction.

Candidate generation and evaluation can run concurrently across trajectories, while candidate processing and state updates within each trajectory proceed sequentially. After the search, the system freezes the best programs saved by all trajectories and jointly trains the final downstream model using their outputs together with the pre-existing structured variables.

5 Experiments

5.1 Experimental Setup

Datasets. We evaluate the proposed method on two public and two private datasets, using models trained only on structured attributes as baselines to measure the incremental gains from text features. The public datasets include CFPB-Timely, constructed from the CFPB Consumer Complaint Database [8], with complaint narratives as text inputs and whether the company responded in a timely manner as the prediction target. It contains 200,000 samples and seven structured features. The other public dataset is a news veracity dataset constructed from the GossipCop subset of Fake-NewsNet [27], with news article text as input alongside structured features such as the number of images. The private datasets, Remark and Report, originate from Qfin Holdings’ credit risk-control applications and contain de-identified text records with corresponding risk labels. Remark comprises remarks made by contacts about target users, whereas Report comprises call records between customer-service agents and users.

Evaluation protocol. We use AUC to evaluate predictive performance. Each dataset is divided into 60% training, 20% validation, and 20% test sets. We conduct independent experiments using five random seeds and report the mean results.

Baselines. Conventional text representation methods include TF-IDF, TF-IDF + SVD, statistical text features (TextStats), and dense embeddings (Embedding). Automated feature engineering methods include OpenFE [31], CAAFE [15], OCTree [21], and LLM-FE [1]. During feature engineering, OCTree uses TextStats extracted from long text as input, while LLM-FE and CAAFE take long text directly as input. We also include FELIX [20] as an LLM-based text feature extraction baseline. Due to the cost of per-sample LLM calls, we

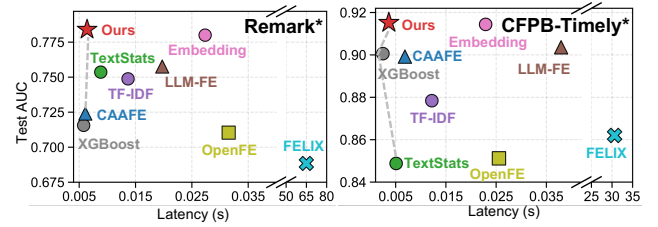


Figure 3: Inference latency and predictive performance across methods.

compare its performance and latency only on subsets of Remark and CFPB-Timely. Except for an additional TF-IDF + Logistic Regression (LR) comparison, all baselines use XGBoost with hyperparameters optimized via Bayesian optimization as the downstream predictor. **Implementation settings.** Each baseline follows its original procedure. LLM-BlockFE and all LLM-based baselines use the locally deployed Seed-OSS-36B-Instruct model [28] with a sampling temperature of $t = 0.1$. LLM-BlockFE uses $K = 5$ search trajectories, provides 20 samples to the LLM per round, and sets the maximum search depth to 5. After the search, the best programs saved by the individual trajectories are frozen for feature extraction. Further dataset details and hyperparameter settings are provided in Appendices A.1 and A.2.

5.2 Offline Performance and Efficiency

Table 1 summarizes the classification performance of LLM-BlockFE and the baseline methods across four datasets. Except for TF-IDF + LR, which uses logistic regression, all feature engineering baselines use XGBoost as the downstream prediction model. LLM-BlockFE achieves the best performance on all datasets, with AUCs of 0.6804, 0.5545, 0.8881, and 0.9472 on the private datasets Remark and Report and the public datasets CFPB-Timely and GossipCop, respectively. These correspond to improvements of 0.0358, 0.0209, 0.0152, and 0.0069 over the strongest baseline on each dataset. These results confirm the method’s generalization ability and superior performance on both private data and general public long text tasks.

Comparing the different baseline paradigms reveals that LLM-based feature engineering methods significantly outperform conventional statistical methods and traditional AutoFE methods overall on the private risk-control datasets, where noise is greater and informative signals are more subtle and complex. This finding clearly demonstrates the stronger capabilities of LLMs to extract deeper semantics and uncover implicit associations in noisy long text. Among the LLM-based methods, LLM-BlockFE further achieves the best performance across all datasets through incremental code-block evolution and credit-guided dynamic backtracking. Moreover, unlike methods that rely on real-time online LLM calls, LLM-BlockFE moves the entire process of constructing complex feature programs to the offline stage. Online deployment requires only sequential execution of the frozen feature code, incurring zero computational or latency overhead from online LLM calls.

Figure 3 compares the classification performance and end-to-end latency of the methods during online inference. To precisely assess inference efficiency, we randomly sample 100 instances from each

Dataset	Base-XGB	TextStats +XGB	TF-IDF +LR	TF-IDF +XGB	TF-IDF+SVD +XGB	Embedding +XGB	OpenFE	CAAFE	OCTree	LLM-FE	Ours
Remark	0.6265	0.6146	0.6058	0.6323	0.6261	0.5685	0.6439	0.6314	0.6435	<u>0.6446</u>	0.6804
Report	0.5323	0.5245	0.5232	0.5314	0.5227	0.5125	0.5250	<u>0.5336</u>	0.5159	0.5324	0.5545
CFPB-Timely	0.8571	0.8418	0.8699	0.8663	0.8707	0.8717	0.8541	0.8684	0.8701	<u>0.8729</u>	0.8881
GossipCop	0.9145	0.9224	0.8603	0.9380	<u>0.9403</u>	0.9330	0.9206	0.9257	0.9206	0.9242	0.9472

Table 1: AUC comparison across datasets. Best results are highlighted in bold, and second-best results are underlined.

full test set to construct the latency benchmark subsets Remark* and CFPB-Timely*. LLM-BlockFE achieves the highest AUC on both benchmark subsets while maintaining millisecond-level inference latency.

Compared with FELIX, a strong baseline that also focuses on efficiency, LLM-BlockFE achieves higher AUC while reducing inference latency by orders of magnitude. This substantial advantage stems from complete architectural decoupling: LLM-BlockFE confines the LLM’s complex logical understanding and feature search processes entirely to the offline stage. Online deployment requires only sequential execution of the frozen programs for feature extraction, eliminating the substantial computational overhead and latency bottleneck of online LLM inference for individual samples.

5.3 Production Deployment and Post-Launch Impact

LLM-BlockFE has been deployed in five real-world financial risk-control applications. During online inference, the system executes feature programs generated and frozen offline, without calling an LLM. Because this feature engineering process does not alter the original structured data, the production system can compute results for two approaches in parallel on exactly the same online samples: one retains the existing manually designed strategy, while the other uses text features generated by LLM-BlockFE. Both approaches use the same samples, labels, structured features, downstream model, and other business strategies. Their performance differences therefore reflect the impact of changing the text feature approach.

Figure 4 shows the monthly KS values of both approaches across the five applications from September to November 2025, covering 2,694,342 real users. Across all 15 application-month comparisons, the LLM-BlockFE approach achieves higher KS than the existing manually designed strategy, with absolute improvements of 0.02–1.56 percentage points. These production results show that, with all other inputs and modeling procedures held fixed, incorporating text features generated by LLM-BlockFE yields higher discrimination in all five risk-control applications.

The system’s practical business value is further validated through real-world online A/B testing. In an online evaluation covering approximately RMB 1 billion in loan disbursements, the treatment group significantly reduced the bad debt rate from 8.0% to 7.7% while keeping the number of approved applicants unchanged. The company has developed more than **200** effective feature chains using this framework, achieving comprehensive coverage of nearly all customer segments. From an engineering perspective, feedback indicates that the deployed feature computation pipelines incur

Setting	AUC	AUC Drop
Base-XGB	0.6190	0.0570
w/o Append-only Constraint	0.6357	0.0403
w/o Bayesian Optimization	0.6383	0.0377
w/o Downstream AUC Gate	0.6268	0.0492
w/o Backtracking	0.6470	0.0290
w/o Credit Guidance	0.6573	0.0187
w/o Depth Calibration	0.6640	0.0120
w/o Cross-Program Sharing	0.6708	0.0052
Full Method	0.6760	0.0000

Table 2: Ablation study on the Remark dataset. AUC Drop denotes the absolute performance decrease relative to the full method.

very low online overhead and require no GPUs, demonstrating strong potential for broader industrial adoption.

5.4 Ablation Study

Table 2 summarizes the ablation results on Remark. Each setting was evaluated over a separate set of five independent runs, distinct from those used for the main comparison in Table 1. The reported AUC is the mean across these runs. The complete method achieves an AUC of 0.6760, an absolute improvement of 0.0570 over Base-XGB. Removing any module reduces AUC, indicating that each design contributes to the final performance under the current experimental setting. Removing the downstream AUC gate has the largest impact: AUC drops to 0.6268, which is 0.0492 below the complete method and only 0.0078 above Base-XGB. This shows that even if a candidate program executes successfully and passes the initial text AUC screening and local parameter optimization, it may not provide a conditional gain to the downstream model. Evaluating the downstream AUC gain for each block helps filter out candidate programs that fail to improve predictive performance.

Ablations of the program construction and search mechanisms also demonstrate the roles of their respective components. Removing the append-only constraint reduces AUC to 0.6357, suggesting that restricting the scope of changes in each round and freezing accepted code blocks helps stabilize program construction. Removing Bayesian optimization reduces AUC to 0.6383, demonstrating the contribution of local parameter optimization to final predictive performance. For the search mechanism, disabling backtracking reduces AUC to 0.6470, indicating that unidirectional expansion is susceptible to constraints imposed by early choices. When credit guidance is removed, the system instead backtracks after a fixed threshold of three failures, and AUC drops to 0.6573. Removing

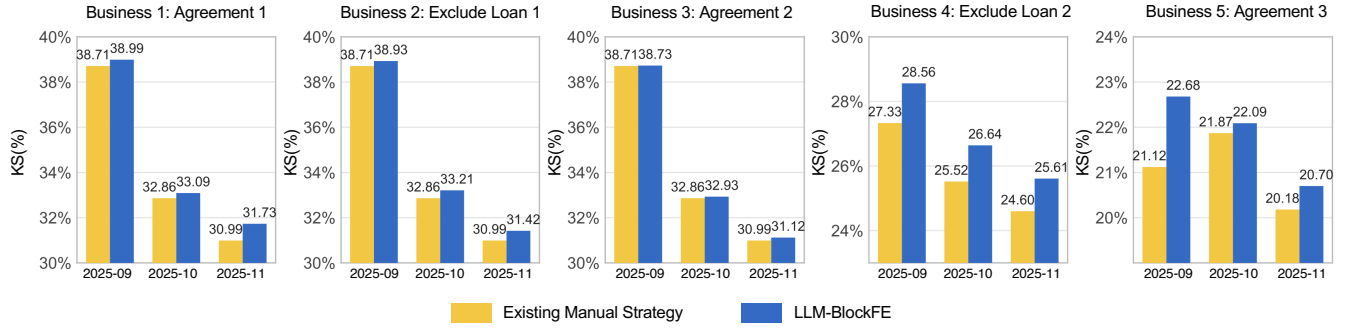


Figure 4: Online performance of LLM-BlockFE across five production risk-control applications. The figure compares monthly KS values for LLM-BlockFE and the existing manually designed strategy on the same online samples from September to November 2025, covering 2,694,342 real-world users.

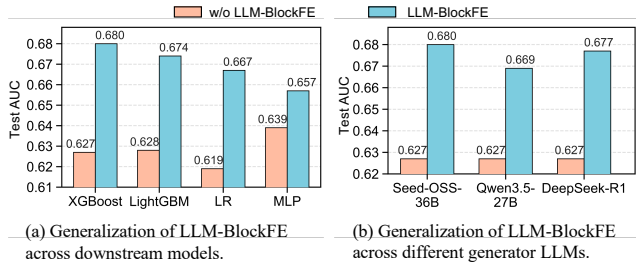


Figure 5: Generalization of LLM-BlockFE across downstream models and generator LLMs on the Remark dataset.

depth calibration reduces AUC to 0.6640. These results indicate that credit guidance provides more effective backtracking decisions than a fixed failure threshold, while depth calibration further improves search outcomes on this dataset. Removing cross-program sharing yields an AUC of 0.6708. Although the decrease is relatively small, it still indicates an additional benefit from sharing exploration directions.

5.5 Generalization Analysis

Generalization across downstream models. Figure 5(a) examines whether the features generated by LLM-BlockFE are effective across different downstream models. Incorporating the generated features improves test AUC for all four models. XGBoost and LightGBM achieve gains of 0.053 and 0.046, respectively, while logistic regression and MLP improve by 0.048 and 0.018. These gains span tree-based, linear, and neural models, suggesting that the generated features provide useful predictive information beyond a single model architecture. The variation in gains also indicates that downstream models differ in how effectively they exploit the generated features.

Generalization across feature-generating LLMs. Figure 5(b) evaluates the sensitivity of LLM-BlockFE to the choice of feature-generating LLM. Relative to the baseline AUC of 0.627 without LLM-BlockFE, using Seed-OSS-36B-Instruct, Qwen3.5-27B [25], and DeepSeek-R1 [10] yields test AUCs of 0.680, 0.669, and 0.677, respectively, corresponding to improvements of 0.053, 0.042, and 0.050.

Dataset	Mean SHAP	Contribution Ratio
Remark	0.3900	41.4%
Report	0.0287	61.1%
CFPB-Timely	0.0235	9.5%
GossipCop	0.6477	19.4%

Table 3: SHAP-based contribution of the features generated by LLM-BlockFE across datasets.

All three generation models improve performance, indicating that the method’s effectiveness does not depend on a single generation model. The differences in gain magnitude nevertheless show that generator choice affects the final predictive performance.

5.6 Feature Contribution and Case Studies

To analyze the role of the textual features generated by LLM-BlockFE in downstream model predictions, Table 3 summarizes their mean absolute SHAP values and relative contribution shares [19]. On the private datasets Remark and Report, the generated textual features account for 41.4% and 61.1% of the total attribution, respectively. On the public datasets CFPB-Timely and GossipCop, the corresponding shares are 9.5% and 19.4%. These results indicate that, under the current experimental settings, the generated textual features account for substantial attribution shares in model predictions on the two private datasets. Together with the AUC improvements obtained by incorporating textual features in Table 1, the model attribution analysis and predictive performance results support the effectiveness of the generated features, suggesting that LLM-BlockFE can construct complementary features from long text that are useful for downstream prediction.

To illustrate how the generated feature is computed, Figure 6 presents a dialogue excerpt, statistics for the full text, and the corresponding code snippet. The complete dialogue contains 57 turns and 1,647 characters across both speakers, with 32 occurrences of the specified discourse particles, yielding a normalized frequency of 0.0194. Rather than applying an adjustment directly based on the occurrence count, the code first divides the count by the text length and then compares the resulting frequency with a threshold. Because this sample’s frequency is below the threshold of 0.0629,

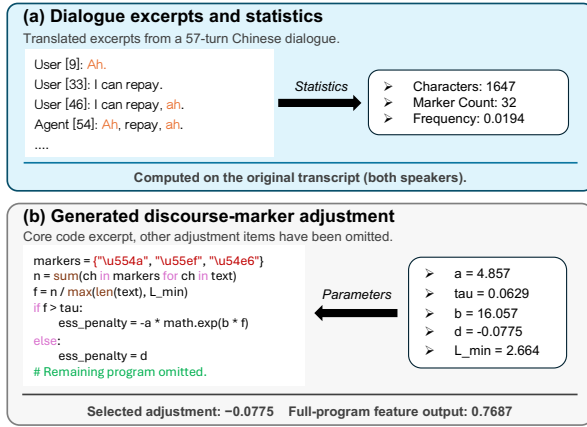


Figure 6: Case study of a deployed feature program on a real customer-service dialogue.

the program follows the default branch and produces an adjustment of -0.0775 , without activating the exponential adjustment branch. Thus, repeated occurrences of discourse particles do not necessarily trigger a stronger negative adjustment. Their effect depends on their frequency relative to the length of the full text. This example makes the relationship between text statistics, conditional logic, and the local numerical adjustment directly traceable.

6 Conclusion

We propose LLM-BlockFE, which formulates feature construction from unstructured long text as an offline search for executable programs. The method builds feature programs by incrementally appending and freezing code blocks. It combines depth-calibrated, credit-guided backtracking with multi-trajectory exploration to reduce dependence on early decisions and limit redundant exploration. The generated programs can be used directly for online feature extraction without further LLM calls, decoupling offline feature discovery from online computation. Experiments show that LLM-BlockFE provides effective complementary features for structured prediction models and achieves the highest AUC in the main experiments. Ablation and generalization experiments further support the contributions of the individual design components and the applicability of the generated features across different downstream models. In five real-world financial risk management applications, post-deployment monitoring shows KS improvements of 0.02 to 1.56 percentage points over existing manually designed strategies. These results suggest that generating inspectable, executable feature programs offline offers a viable approach to leveraging long text information in industrial prediction systems.

References

- [1] Nikhil Abhyankar, Parshin Shojaei, and Chandan K Reddy. 2025. LLM-FE: Automated feature engineering for tabular data with llms as evolutionary optimizers. *arXiv preprint arXiv:2503.14434* (2025).
- [2] Abhinav Agarwal, Yan Shuo Tan, Omer Ronen, Chandan Singh, and Bin Yu. 2022. Hierarchical Shrinkage: Improving the accuracy and interpretability of tree-based models. In *International Conference on Machine Learning*. PMLR, 111–135.
- [3] Vojtěch Balek, Lukáš Šykora, Vilem Sklenák, and Tomáš Kliegr. 2025. LLM-based feature generation from text for interpretable machine learning. *Machine Learning* 114, 11 (2025), 241.
- [4] Merwan Barlier and Blaz Skrlj. 2026. LLMs as Feature Engineers for Text-and-Tabular Prediction. *arXiv preprint arXiv:2609.21894* (2026).
- [5] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).
- [6] Keith Burghardt, Jienan Liu, Sadman Sakib, Yuning Hao, and Bo Li. 2026. FAMOSE: A ReAct Approach to Automated Feature Discovery. *arXiv preprint arXiv:2602.17641* (2026).
- [7] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [8] Consumer Financial Protection Bureau. 2026. Consumer Complaint Database. <https://www.consumerfinance.gov/data-research/consumer-complaints/>. Accessed: 2026-09-30.
- [9] Ziming Dai, Dabiao Ma, Jinle Tong, Mengyuan Han, Jian Yang, Hongtao Liu, Haojun Fei, and Qing Yang. 2026. NSR-Boost: A Neuro-Symbolic Residual Boosting Framework for Industrial Legacy Models. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 7129–7140.
- [10] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948* [cs.CL]. <https://arxiv.org/abs/2501.12948>
- [11] Alvaro Fialho, Marc Schoenauer, and Michèle Sebag. 2010. Toward comparison-based adaptive operator selection. In *Proceedings of the 12th annual conference on Genetic and evolutionary computation*. 767–774.
- [12] Motomasa Fujii, Hiroki Sakaji, Shigeru Masuyama, and Hajime Sasaki. 2022. Extraction and classification of risk-related sentences from securities reports. *International Journal of Information Management Data Insights* 2, 2 (2022), 100096.
- [13] Paul H Garthwaite, Maha W Moustafa, and Fadlalla G Elfadaly. 2024. Locally correct confidence intervals for a binomial proportion: A new criteria for an interval estimator. *Scandinavian Journal of Statistics* 51, 1 (2024), 220–244.
- [14] Sungwon Han, Sungkyu Park, and Seungeon Lee. 2025. Tabular feature discovery with reasoning type exploration. *arXiv preprint arXiv:2506.20357* (2025).
- [15] Noah Hollmann, Samuel Müller, and Frank Hutter. 2023. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *Advances in Neural Information Processing Systems* 36 (2023), 44753–44775.
- [16] Cuiqing Jiang, Lan Ma, Zhao Wang, and Bo Chen. 2023. Financial distress prediction using the Q&A text of online interactive platforms. *Electronic Commerce Research and Applications* 61 (2023), 101292.
- [17] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017).
- [18] Chohee Kim, Mihaela Van Der Schaar, and Changhee Lee. 2024. Discovering Features with Synergistic Interactions in Multiple Views. In *ICML*. 24562–24583.
- [19] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017).
- [20] Simon Malberg, Edoardo Mosca, and Georg Groh. 2024. FELIX: Automatic and interpretable feature engineering using LLMs. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 230–246.
- [21] Jaehyun Nam, Kyuyoung Kim, Seunghyuk Oh, Jihoon Tack, Jaehyung Kim, and Jinwoo Shin. 2024. Optimized feature generation for tabular data via llms with decision tree reasoning. *Advances in neural information processing systems* 37 (2024), 92352–92380.
- [22] Alexander Novikov, Ngán Vū, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislov Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. 2025. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131* (2025).
- [23] Julian Parsert and Elizabeth Polgreen. 2024. Reinforcement learning and data-generation for syntax-guided synthesis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 10670–10678.
- [24] Qianqian Qi, David J Hessen, Tejaswini Deoskar, and Peter GM van der Heijden. 2024. A comparison of latent semantic analysis and correspondence analysis of document-term matrices. *Natural Language Engineering* 30, 4 (2024), 722–752.
- [25] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- [26] Parshin Shojaei, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan Reddy. 2025. LLM-SR: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations*, Vol. 2025. 16054–16085.
- [27] Kai Shu, Deepak Mahudeswaran, Suhang Wang, Dongwon Lee, and Huan Liu. 2020. Fakenewsnet: A data repository with news content, social context, and spatiotemporal information for studying fake news on social media. *Big data* 8, 3 (2020), 171–188.
- [28] ByteDance Seed Team. 2025. Seed-OSS Open-Source Models. <https://github.com/ByteDance-Seed/seed-oss>.
- [29] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with

large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.

- [30] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. 2020. Big bird: Transformers for longer sequences. *Advances in neural information processing systems* 33 (2020), 17283–17297.
- [31] Tianping Zhang, Zheyu Aqa Zhang, Zhiyuan Fan, Haoyan Luo, Fengyuan Liu, Qian Liu, Wei Cao, and Li Jian. 2023. OpenFE: Automated feature generation with expert-level performance. In *International Conference on Machine Learning*. PMLR, 41880–41901.

A Appendix

A.1 Dataset Details

Table 4 summarizes all datasets used in our evaluation. The public datasets are primarily derived from open platforms such as Kaggle, while the private industrial datasets are provided by Qfin Holdings. Since high-stakes applications such as financial risk control, which are the focus of this paper, are typically formulated as binary classification tasks, our evaluation focuses uniformly on this setting. To meet the training requirements of the downstream base models and rigorously quantify the performance gains achieved by our method, we specifically select and reconstruct multimodal datasets that contain both initial structured attributes and unstructured long text.

Specifically, we construct the CFPB-Timely dataset from the publicly available CFPB Consumer Complaint Database. This dataset records detailed consumer complaints about various financial products, along with the companies’ handling statuses. For our task, we use the “complaint narrative” as the unstructured input and whether the company responded in a timely manner as the binary prediction target. We also retain seven basic structured features available when the complaint is received, such as product type, sub-product type, and issue category. For the GossipCop dataset, we use the “news article text” as the unstructured long text input and news veracity (real or fake) as the prediction label. In addition, we extract statistical metadata associated with each news article, including the number of images, number of shares, time of dissemination, and intervals between shares, as the dataset’s initial structured attributes.

Dataset	Structured Features	Unstructured Long Text	Samples	Source
CFPB-Timely	7	1	200,000	Kaggle
GossipCop	7	1	6,344	Kaggle
Remark	32	1	100,000	Qfin
Report	5	1	100,000	Qfin

Table 4: Dataset statistics.

Furthermore, to evaluate the model in real-world credit risk-control applications, we include two private datasets. The Report dataset consists of call records between the company’s customer service agents and users. The Remark dataset extracts remarks about target users made by their contacts by parsing authorized address books and using data shared by third-party anti-fraud alliances. Both datasets contain de-identified text and semi-structured records, enabling a comprehensive evaluation of the model’s feature extraction capabilities on complex industrial data.

Our dataset selection spans settings from small to large scale. With varied feature dimensionalities and sample sizes, this diverse collection represents a broad range of real-world scenarios and provides a robust framework for evaluating the proposed method.

A.2 Hyperparameter Settings

Table 5 reports the hyperparameter settings used by LLM-BlockFE. We use a window length of 12, a prediction horizon of 3 attempts, a prior strength of 4, a rollback threshold of 0.5, and a default success rate of 0.1.

Symbol	Description	Value
W	Window length	12
H	Prediction horizon (attempts)	3
κ	Prior strength	4
ρ	Rollback threshold	0.5
p_0	Default success rate	0.1

Table 5: Hyperparameter settings.

A.3 Experimental Stability Analysis

We run each method independently five times on each dataset and compute the standard deviation of test AUC to assess experimental stability. As shown in Table 6, the standard deviations of LLM-BlockFE on Remark, Report, CFPB-Timely, and GossipCop are 0.0046, 0.0088, 0.0062, and 0.0063, respectively. The results show that variability across runs differs across datasets, with relatively greater variability on Report. Overall, its standard deviations are broadly comparable to those of the LLM-based baselines, indicating that the observed performance is not attributable to a single run.

Dataset	Base-XGB	TextStats +XGB	TF-IDF +LR	TF-IDF +XGB	TF-IDF+SVD +XGB	Embedding +XGB	OpenFE	CAAFE	OCtree	LLM-FE	Ours
Remark	± 0.0013	± 0.0026	± 0.0000	± 0.0013	± 0.0038	± 0.0031	± 0.0028	± 0.0071	± 0.0020	± 0.0065	± 0.0046
Report	± 0.0065	± 0.0046	± 0.0000	± 0.0043	± 0.0037	± 0.0069	± 0.0035	± 0.0144	± 0.0101	± 0.0024	± 0.0088
CFPB-Timely	± 0.0049	± 0.0023	± 0.0000	± 0.0012	± 0.0007	± 0.0016	± 0.0050	± 0.0063	± 0.0034	± 0.0051	± 0.0062
GossipCop	± 0.0008	± 0.0013	± 0.0000	± 0.0017	± 0.0009	± 0.0018	± 0.0014	± 0.0046	± 0.0019	± 0.0046	± 0.0063

Table 6: The standard deviation of AUC values obtained from five runs on different datasets.